Modern Reinforcement Learning: From Agents to LLMs

Function Approximation: From Linear Models to Deep RL

Lecture 3

Keywords: linear/nonlinear FA, semi-gradient TD, convergence, deadly triad, stabilization methods

Ilija Bogunovic

Today's Roadmap

1) Motivation & Recap – Why we need function approximation in RL

2) Value Function Approximation – Linear models, semi-gradient TD, convergence

3) Instability: The Deadly Triad – When FA + bootstrapping + off-policy diverge

4) Stabilizing Methods – Target networks, replay buffers, double estimators, regularization

5) Connecting to Modern RL for LLMs – Large models as function approximators

📘 **Side note / "Keep in mind" box:**
- Each section explains how approximating value functions changes the behavior of Bellman updates.

Why Tabular RL Fails in Real Problems

— State spaces in real tasks are enormous or continuous

— Tabular value functions require one entry per state (or per state–action)

— No generalization: every state must be visited many times

— Memory + sample cost grow exponentially

→ We need parametric value functions to generalize across states

📘 Even modest environments (e.g., Atari with 84×84×4 observations) have tens of thousands of dimensions → tabular RL is impossible.

State Spaces Grow Exponentially

— Many RL tasks have **high-dimensional** or **continuous state** spaces

— Representing every state explicitly is impossible

— Sample complexity also grows exponentially → naive RL needs countless visits

— Even modest problems (vision, robotics, language) have millions of possible states

→ RL must **generalize** from limited experience

Targets of Approximation in RL

- The key quantities in RL are **functions**, not tables:
  - Value functions $V^\pi(s)$
  - Action-value functions $Q^\pi(s, a)$
  - Policies $\pi(a \mid s)$
- In large state spaces, each must be represented **compactly**
- Most RL algorithms rely on value estimates → the bottleneck is approximating V and Q
- Approximation = representing these functions with a finite parameter vector θ

Approximation Is Everywhere in Modern RL

— All practical RL systems rely on **parameterized function approximators**

— Deep RL: value functions and policies represented by neural networks

— Control tasks, vision-based RL, model-based RL → all use approximation

— LLMs: massive function approximators used for sequential decision-making

→ Understanding value approximation is essential for scaling RL

Value Function Approximation

Value Function Approximation: Formal View

— Replace true value function with a parameterized estimate

$$\hat{V}_\theta(s) \approx V_\pi(s)$$
$$\hat{Q}_\theta(s, a) \approx Q_\pi(s, a)$$

— Parameters θ define the function's shape

— Goal: choose θ to minimize approximation error

— Enables compact representation + generalization over states

— Two main classes: **linear** and **nonlinear** approximators

Linear Value Approximation

— Represent each state by a **feature vector** $\phi(s) \in \mathbb{R}^d$

— Approximate value function as a **linear combination of features**:

$$\hat{V}_\theta(s) = \theta^\top \phi(s)$$

— Parameters $\theta \in \mathbb{R}^d$ control contribution of each feature

— Simple, efficient, convex optimization landscape

— Foundation for convergence guarantees (coming next)

Monte Carlo Value Regression

— MC target: full return

— Objective: minimize squared prediction error

— Gradient descent update:

— Works like supervised regression with input = features, label = return

— Stable and unbiased, but **high variance**

Bootstrapped Value Learning: Semi-Gradient TD(0)

- Use one-step bootstrapped target:

- Semi-gradient TD(0) update:

- "Semi-gradient" because the target **depends on** θ but we **do not differentiate through it**
- Lower variance than MC → faster learning
- But introduces bias → can be unstable under certain conditions

Convergence of Linear TD(0): On-Policy Case

- Under linear function approximation

$$V_\theta(s) = \phi(s)^\top \theta$$

Convergence of Linear TD(0): On-Policy Case

- Under linear function approximation

$$V_\theta(s) = \phi(s)^\top \theta$$

the expected TD update satisfies

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t)$$

Convergence of Linear TD(0): On-Policy Case

- Under linear function approximation

$$V_\theta(s) = \phi(s)^\top \theta$$

the expected TD update satisfies

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t)$$

where

$$A = \mathbb{E}\left[\phi(s_t)(\phi(s_t) - \gamma\phi(s_{t+1}))^\top\right]$$

$$b = \mathbb{E}[r_{t+1}\phi(s_t)]$$

Convergence of Linear TD(0): On-Policy Case

- Under linear function approximation

$$V_\theta(s) = \phi(s)^\top \theta$$

the expected TD update satisfies

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t)$$

where

$$A = \mathbb{E}\left[\phi(s_t)(\phi(s_t) - \gamma\phi(s_{t+1}))^\top\right]$$

$$b = \mathbb{E}[r_{t+1}\phi(s_t)]$$

Interpretation:

- TD is trying to solve the **linear system** $A\theta = b$ whose solution is $\theta^* = A^{-1}b$

Why TD(0) Works On-Policy

- Under **on-policy sampling**, states follow the stationary distribution d_π

Why TD(0) Works On-Policy

- Under **on-policy sampling**, states follow the stationary distribution d_π
- Using this distribution, the matrix A can be written as

$$A = \Phi^\top D_\pi (I - \gamma P_\pi)\Phi$$

- where
  - Φ : feature matrix
  - D_π : diagonal matrix with stationary distribution
  - P_π : transition matrix under policy π

Why TD(0) Works On-Policy

- Under **on-policy sampling**, states follow the stationary distribution d_π
- Using this distribution, the matrix A can be written as

$$A = \Phi^\top D_\pi (I - \gamma P_\pi)\Phi$$

- where
  - Φ : feature matrix
  - D_π : diagonal matrix with stationary distribution
  - P_π : transition matrix under policy π
- **Key property:** Under on-policy sampling $A \succ 0$ so A is **positive definite**
- **Consequence:** The system $A\theta = b$ has a **unique solution** $\theta^* = A^{-1}b$

Linear TD(0) Converges

- The expected TD update is

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t),$$

this is a **stable linear recursion** when $A \succ 0$

Linear TD(0) Converges

- The expected TD update is

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t),$$

this is a **stable linear recursion** when $A \succ 0$

- With stochastic samples the update becomes

$$\theta_{t+1} = \theta_t + \alpha_t(b - A\theta_t + \text{noise}_t)$$

Linear TD(0) Converges

- The expected TD update is

$$\theta_{t+1} = \theta_t + \alpha(b - A\theta_t),$$

this is a **stable linear recursion** when $A \succ 0$

- With stochastic samples the update becomes

$$\theta_{t+1} = \theta_t + \alpha_t(b - A\theta_t + \text{noise}_t)$$

- Stochastic approximation result

$$\sum_t \alpha_t = \infty \quad \text{and} \quad \sum_t \alpha_t^2 < \infty$$

then $\theta_t \to \theta^*$ with probability 1.

From Linear Models to Neural Networks

— Linear FA:

- Convex
- Stable
- Well-understood

— But linear models often **cannot represent** complex value functions

— Nonlinear FA (e.g., neural networks):

$$\hat{V}_\theta(s) = f_\theta(\phi(s))$$

— Pros: expressive, can approximate highly nonlinear value landscapes

— Cons: non-convex loss surfaces → **no general convergence guarantees**

Bias–Variance Trade-off Under Nonlinear FA

- MC targets:
  - Unbiased
  - High variance
  - More stable with nonlinear models
- TD targets:
  - Lower variance
  - But biased because the target depends on current θ
- With nonlinear FA, TD's bias + bootstrapping can lead to **unpredictable behavior**
- Practical pattern:
  - MC → stable but slow
  - TD → fast but can diverge

Instability: The Deadly Triad

Why TD + Function Approximation Can Diverge

- TD(0) convergence relied on: 1) **Linear** function approximation; 2) **On-policy** sampling; 3) **Bootstrapping structure** being well-behaved

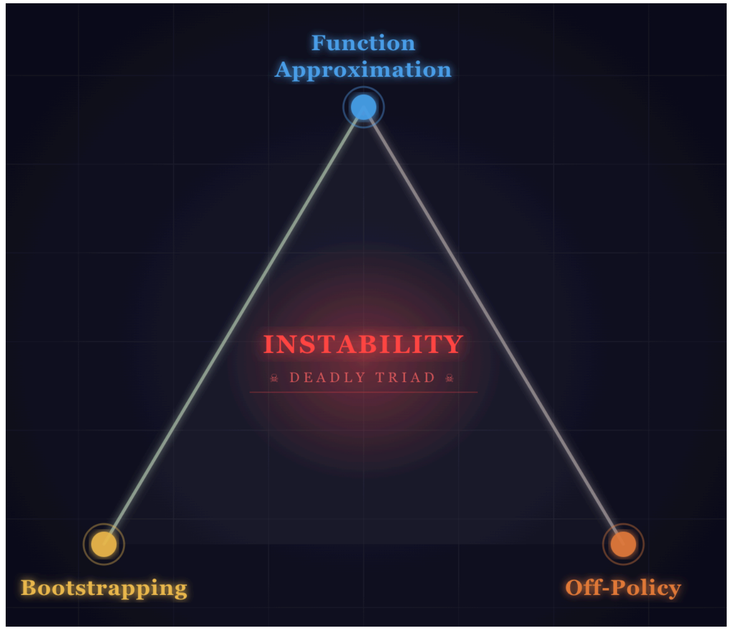
- Violating these assumptions → the matrix A may not be positive definite

- With nonlinear FA or off-policy data:
  - No contraction property
  - No convexity
  - Error can amplify

- Result: TD updates may **oscillate** or **diverge**

The Deadly Triad: Three Ingredients That Cause Divergence

- Divergence often arises when **three factors** appear together:
  1) Function Approximation
  2) Bootstrapping (e.g., TD, n-step updates)
  3) Off-Policy Learning
- Each factor alone is usually manageable
- Any **two** can still be stable (in many cases)
- But **all three together** can create uncontrollable feedback loops
- This is the central explanation for instability in many RL algorithms



Why Function Approximation Breaks TD Guarantees

- In the tabular setting, TD learning behaves well:
  - Updates move values toward the Bellman target
  - The process converges to the correct value function
- With function approximation, we restrict values to a parameterized form:

$$V_\theta(s) = \phi(s)^\top \theta$$

- Now updates at one state affect many other states because parameters are shared
- The true value function may not lie inside the function class
- Consequence:
  - Updates may not reduce errors
  - In some cases, value estimates can diverge
- This is a key reason why TD + function approximation can become unstable

Bootstrapping Can Magnify Small Mistakes

- TD methods use predicted values as part of the target:

$$y_t = r_t + \gamma \hat{V}_\theta(s_{t+1})$$

- If $\hat{V}_\theta(s_{t+1})$ is inaccurate, the error is **fed back** into the next update
- With function approximation, these errors propagate **across many states** via shared parameters
- Result:
  - Errors accumulate
  - TD update may overshoot
  - Parameter updates may diverge if the projection does not control them
- Bootstrapping is efficient — but fragile under approximation

Off-Policy Sampling Breaks Stability

- Off-policy methods learn about a **target policy** while sampling from a **behavior policy**
- This breaks the stationary distribution used in the linear TD convergence proof
- Feature expectations no longer match the transition dynamics of the target policy
- As a result:
  - The matrix A can lose positive definiteness
  - TD updates may push parameters in the wrong direction
- Off-policy + FA + bootstrapping → highly unstable (Deadly Triad)

A Simple MDP Where Off-Policy TD(0) Must Diverge

Summary: When Instability Arises

When Do Value-Function Methods Become Unstable?

- Instability occurs when the 3 components of the Deadly Triad interact:
  1) **Function approximation** (shared parameters link states)
  2) **Bootstrapping** (predictions depend on predictions)
  3) **Off-policy sampling** (updates from the wrong distribution)
- Consequences:
  - Updates no longer guarantee that errors shrink
  - Errors are fed back and amplified
  - Updates may drift away from the true value

Key message:

Without additional stabilization, TD-style methods with FA **are not guaranteed to converge**.

Deep RL: Same Structure, Bigger Models, Same Problems

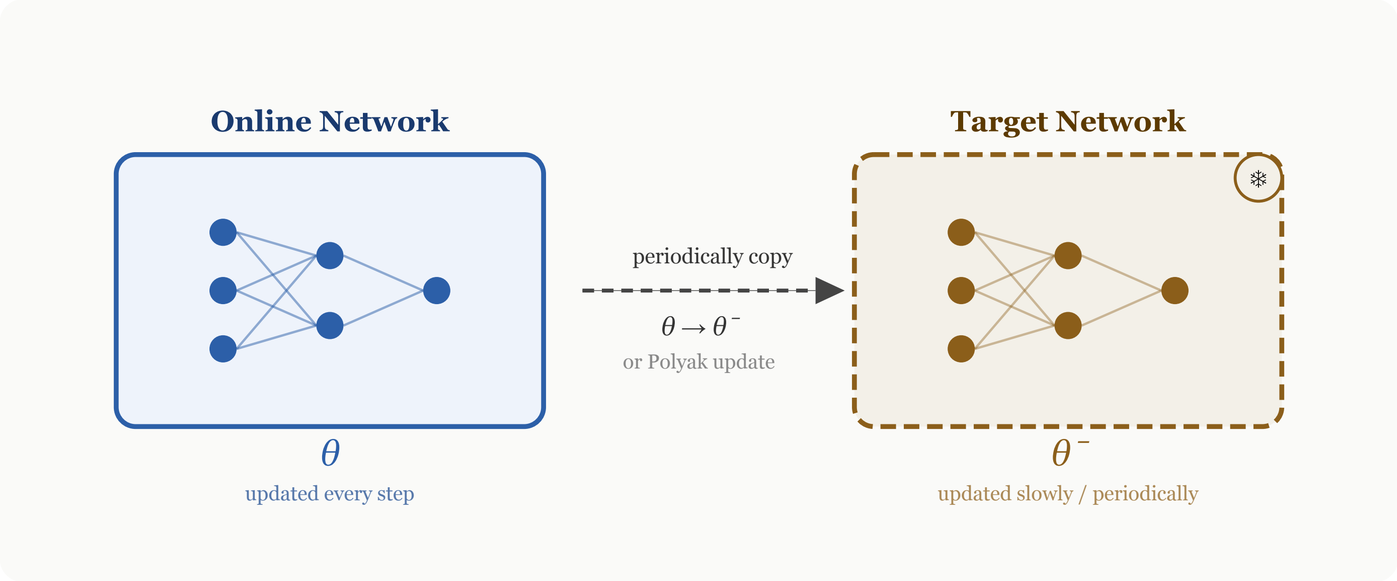
- Deep RL uses **nonlinear neural networks** for value and policy approximation
- Nonlinear FA breaks convexity → stability is even harder to guarantee
- Bootstrapping in TD and Q-learning is still central
- Off-policy data is common (experience replay, behavioral cloning, offline RL)
- Therefore:
  - Deep RL inherits the exact structure of the Deadly Triad
  - Instability is often worse (nonlinear dynamics, high-dimensional parameters)

Key takeaway: Deep RL didn't "solve" the triad — it just added more powerful approximators and new stabilization mechanisms.

Stabilization Methods

Target Networks: Stabilizing Bootstrapping

- Core idea: use a separate, slow-moving value function for TD targets
- Replace: $y_t = r + \gamma \hat{V}_\theta(s')$ with $y_t = r + \gamma \hat{V}_{\theta^-}(s')$ where θ^- is a **frozen** or **slowly updated** copy
- Prevents harmful feedback loops from rapidly changing estimates
- Essential in DQN and nearly all deep TD-based RL algorithms
- Effectively restores approximate contraction by limiting target drift



Experience Replay: Breaking Correlations & Smoothing Learning

- Store past transitions (s, a, r, s') in a **replay buffer**
- Sample mini-batches uniformly (or with priorities) during updates
- Benefits:
  - Breaks correlations in sequential data
  - Provides approximately i.i.d. samples → more stable gradients
  - Allows safer off-policy updates (same transition reused many times)
  - Reduces variance in TD targets by smoothing over diverse experiences
- Critical component of DQN, DDQN, TD3, SAC

Overestimation & Double Q-learning

- Standard Q-learning uses:

$$y = r + \gamma \max_a Q_\theta(s', a)$$

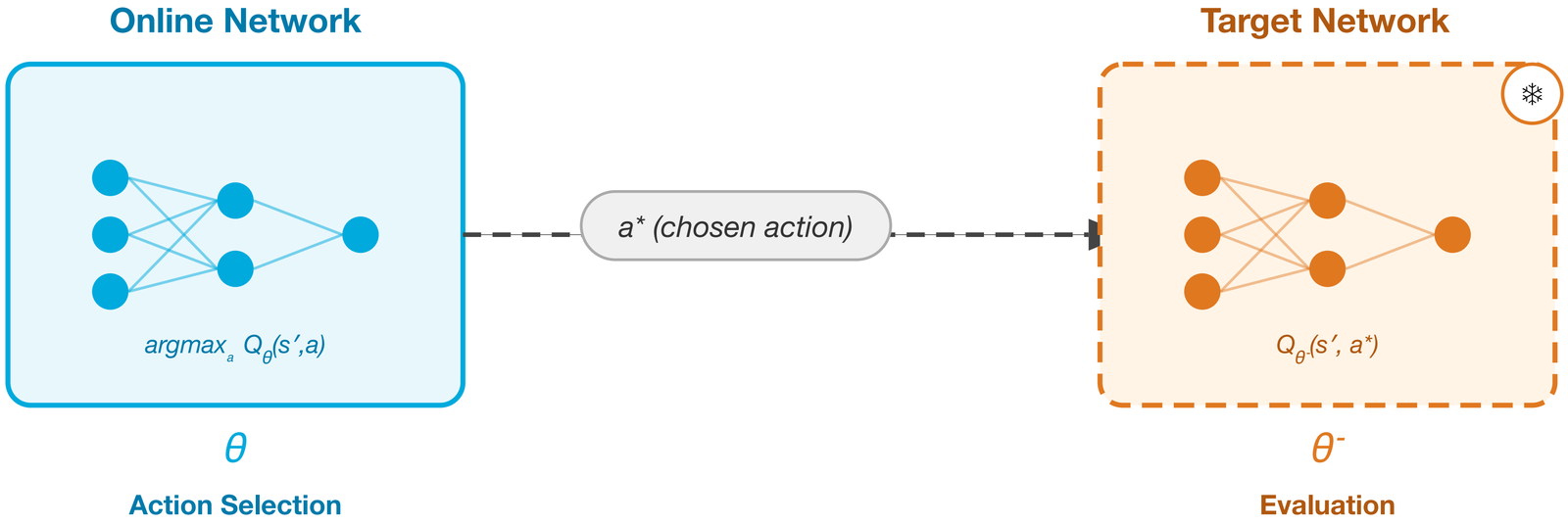
- The **max operator** over noisy estimates induces systematic overestimation
  - Especially harmful with function approximation
  - Leads to runaway Q-values & unstable learning

Overestimation & Double Q-learning

- Standard Q-learning uses:

$$y = r + \gamma \max_a Q_\theta(s', a)$$

- The **max operator** over noisy estimates induces systematic overestimation
  - Especially harmful with function approximation
  - Leads to runaway Q-values & unstable learning



$y = r + \gamma \cdot Q_{\theta^-}(s', a^*)$ where $a^* = \text{argmax}_a\, Q_\theta(s', a)$

Overestimation & Double Q-learning

- Standard Q-learning uses:

$$y = r + \gamma \max_a Q_\theta(s', a)$$

- The **max operator** over noisy estimates induces systematic overestimation
  - Especially harmful with function approximation
  - Leads to runaway Q-values & unstable learning

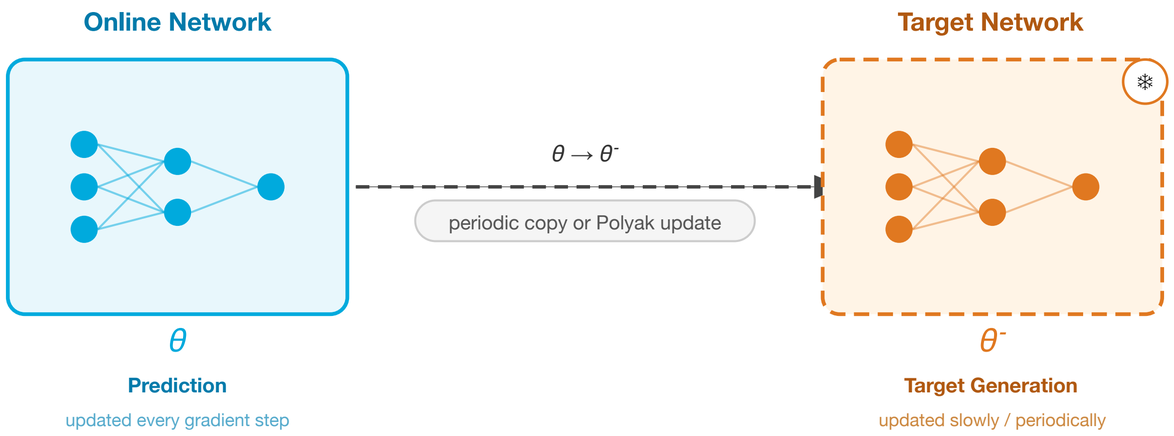
- **Double Q-learning idea:**
  - Use one network for **action selection**
  - Use the other for **evaluation**

$$y = r + \gamma Q_{\theta^-}(s', \arg\max_a \; Q_\theta(s', a))$$

- Deep RL variant → Double DQN
- Significantly reduces bias → more stable training

Deep RL Trick: Separating Target and Online Networks

- In deep RL, it is standard to maintain two networks:
  - **Online network** (θ): updated every gradient step
  - **Target network** (θ^-): used only for computing TD targets
- Why this matters:
  - Prevents rapid feedback loops (targets stay stable) and helps approximate fixed-point iteration
  - Reduces oscillations in TD updates
- Used in: DQN, DDQN, TD3, SAC, Actor–Critic critics; Combined with replay buffer → core ingredients of stable deep Q-learning



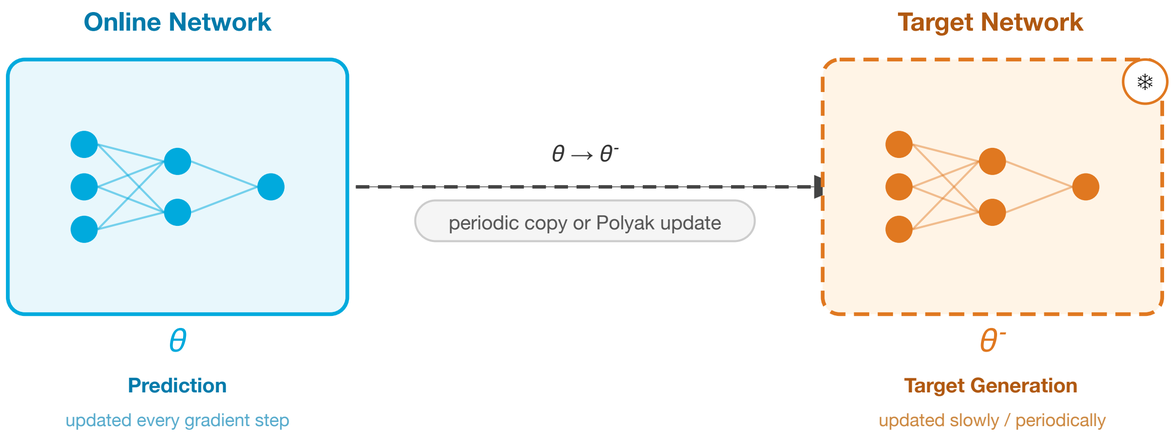
Averaging (Polyak / EMA) Targets: Smoother Stabilization

- Instead of copying the online network every K steps, maintain a moving average:

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$$

- Advantages:
  - Targets evolve **slowly and smoothly**
  - Reduces oscillations and large jumps in TD targets
  - More stable than periodic hard updates

- Used heavily in: TD3, SAC, many actor–critic variants; Provides better approximation of a **fixed point iteration**



Trust Regions & KL Regularization

- Policy updates can destabilize value learning if they change too fast
- **Trust region idea:** restrict updates to stay close to the old policy
- In practice: enforce a **KL divergence constraint**

$$KL(\pi_{new} \parallel \pi_{old}) \leq \delta$$

- Implemented via:
  - **TRPO (explicit trust region)**
  - **PPO (clipped surrogate objective)**
  - **KL penalty** in GRPO / other modern methods
- Effect: prevents large policy shifts that destabilize bootstrapped critics

Normalization: Stabilizing Value & Policy Networks

- Neural networks in RL are highly sensitive to scale of inputs
- **Input normalization:**
  - Standardize observations (mean 0, variance 1)
  - Makes features behave more linearly → reduces distortion in FA
  - Common in robotics, continuous control (e.g., MuJoCo tasks)
- **Batch normalization** (less common but sometimes used):
  - Stabilizes intermediate activations
  - Can help critics generalize better
- Overall effect:
  - Reduces gradient explosion
  - Prevents value-function magnitudes from drifting
  - Helps keep TD targets within predictable ranges

Regularization: Controlling Parameter Growth

- Regularization helps prevent unstable parameter updates
- **Weight decay (L2 regularization):**
  - Penalizes large weights
  - Encourages smoother value functions
  - Reduces sensitivity to bootstrapping noise
- **Constraints & clipping:**
  - Parameter norm constraints
  - Gradient clipping to prevent large TD-induced jumps
- Effect:
  - Dampens error amplification
  - Helps maintain numerical stability
  - Often essential for critic networks in actor–critic methods

Practical Stabilization Guidelines

- **Always** use a target network for TD-based value learning
- Use **experience replay** unless the method is strictly on-policy
- Prefer **Double Q-learning / Double DQN** to avoid overestimation bias
- Use **EMA / Polyak** averaging for smoother target updates
- Apply **observation normalization** (and LayerNorm if needed)
- Add **regularization**: weight decay + gradient clipping
- Keep **learning rates conservative** — value networks are sensitive
- Constrain policy updates via **KL penalties or clipping methods** (PPO-style)
- **Key message:**
  These techniques collectively restore approximate contraction and help prevent divergence.

From Stable Function Approximation to Stable Deep RL

- Deep RL = classical RL + nonlinear function approximators (neural nets)
- Stability depends on controlling:
  - Bootstrapped targets
  - Value estimation drift
  - Off-policy data influence
- The stabilization methods you've seen:
  - **Target networks** approximate fixed-point iteration
  - **Replay buffers** approximate i.i.d. sampling
  - **Double Q-learning** reduces bias
  - **Averaging / EMA** smooths updates
  - **Normalization + regularization** reduce error amplification
  - **KL constraints** keep policy changes small
- Combined, they recreate a **near-contractive** update process
- Core idea: stable value approximation is the foundation for stable deep RL

Outlook

LLMs as Enormous Parametric Policies

- An LLM defines a large parametric policy:

$$\pi_\theta(a \mid s) = \text{distribution over next tokens given context}$$

- Same structure as a policy in RL:
  - **state** = token sequence / context / prompt / question
  - **action** = next token / answer
- LLMs are extremely large function approximators (billions of parameters)
- Learning with LLMs = updating θ to improve sequential decisions

→ RL concepts (value approx., policy improvement) naturally reappear

LLM Training Shares RL's Stability Challenges

- LLMs are **huge nonlinear function approximators** → same stability issues as deep RL
- RL-style updates appear in modern LLM training:
  - **Bootstrapping** in chain-of-thought (later steps rely on earlier predictions)
  - **Off-policy data** from large logged datasets (web corpus, instructions, preferences)
  - **Very large parameter spaces** → small errors can propagate widely
- Therefore, training LLMs with RL-like objectives requires stable value/policy approximation, just like classical RL
- Many deep RL stabilizers have direct analogues in LLM training (e.g., KL control, target policies, normalization)

Why RL Fundamentals Inform Stable LLM Training

- Stability:
  - LLMs are massive nonlinear approximators → susceptible to the same instability as deep RL critics
  - Small errors can propagate through long token sequences
- Projection interpretations:
  - LLM training often uses constrained updates (e.g., KL penalties)
  - Analogous to RL's trust regions → keep updates in "safe" regions where approximation is reliable
- Regularization parallels:
  - Weight decay, normalization, and conservative learning rates are used in LLMs for the same reasons as in deep RL
  - These help control error amplification in sequential predictions
- Big picture:
  - Core RL ideas around stable value/policy approximation transfer directly to reasoning, alignment, and post-training of LLMs

Summary

Summary: Key Risks in RL + Tools to Handle Them

- **Risk 1: Instability from function approximation**
  - Nonlinear approximators + TD bootstrapping → potential divergence
- **Risk 2: Off-policy drift**
  - Mismatch between behavior and target distributions destabilizes learning
- **Risk 3: Error amplification**
  - Bootstrapped targets propagate approximation errors across states
  - Stabilization tools we covered:
    - Target networks (hard or EMA)
    - Experience replay
    - Double estimators
    - KL constraints / trust regions
    - Normalization & regularization
    - Conservative learning rates
- **Takeaway:** Stable RL = controlling approximation, feedback loops, and policy drift.

From RL Fundamentals to Deep RL to LLMs

- **Classical RL theory (Bellman equations, contraction, TD, FA)**
  - Provides the mathematical foundations
  - Explains when learning is stable vs. unstable
- **Modern Deep RL**
  - Adds neural networks → nonlinear FA
  - Requires stabilization: target nets, replay, KL constraints, etc.
  - Still governed by the same principles from classical RL
- **LLM training & alignment**
  - LLM ≈ massive policy/value approximator
  - Faces the same stability challenges: bootstrapping, off-policy data, drift
  - Methods like KL penalties, conservative updates, and normalization directly mirror RL stabilizers

Next Lecture: Policy Gradient Methods

- Thus far: value-function methods + function approximation
- Next: directly optimize **parameterized policies**
- Key ideas of Lecture 5:
  - Derive the **policy gradient theorem**
  - Understand **REINFORCE** and variance reduction
  - Link policy gradients → actor–critic methods
  - See how value approximation and policy optimization interact
- Opens the door to modern RL algorithms (PPO, SAC, TRPO, etc.)

Recommended Reading

- Sutton & Barto (2nd Ed.)
  - Ch. 9 — On-policy Prediction with Approximation
    - Linear value function approximation
    - Semi-gradient TD methods
    - Projected Bellman equation
  - Ch. 11 — Off-policy Methods with Approximation
    - Instability, the deadly triad
    - Baird's counterexample
    - Why off-policy TD can diverge
  - Ch. 6.7 — Double Learning
    - Foundation for Double Q-learning
  - Ch. 10.2 — Overestimation Bias

- Deep RL (optional, modern context)
  - Mnih et al., Human-level Control through Deep RL (2015) — DQN
  - van Hasselt et al., Deep Reinforcement Learning with Double Q-learning (2016)
  - Lillicrap et al., DDPG (2015) — Polyak averaging
  - Fujimoto et al., TD3 (2018) — Target smoothing & EMA
  - Schulman et al., PPO (2017) — KL & clipping